

Parallel Computing

BUDURA Bogdan, DESHORS Victor

March 18, 2025

Outline

- 1 Repository Overview
- 2 OpenMP Strategy
- 3 CUDA Strategy
- 4 MPI and Hybrid Strategies
- 5 Hybrid CUDA+MPI+OpenMP
- 6 Conclusion

Branches:

- Using_Only_GPU
- Using_Only_MPI
- Using_Only_OpenMP
- Using_OpenMP_MPI_CUDA

Components:

- Image folders
- Source code (src/, src_para/)
- Compilation artifacts (obj/, obj_para/)
- Execution scripts
- Performance tracking CSV files

OpenMP Parallelization

Parallelized functions:

- `load_pixels`
- `apply_gray_filter`
- `apply_blur_filter`
- `apply_sobel_filter`

Not Parallelized:

- `store_pixels`

OpenMP Performance

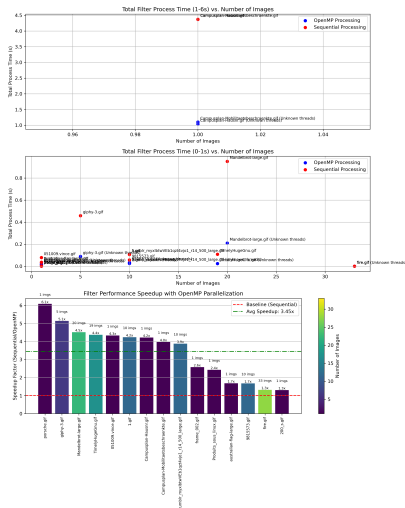


Figure: Runtime OpenMP Version

Implemented for:

- `apply_blur_filter`
- `apply_sobel_filter`

Key optimizations:

- Asynchronous execution
- Optimized thread blocks (16×16)
- Memory management

CUDA Performance

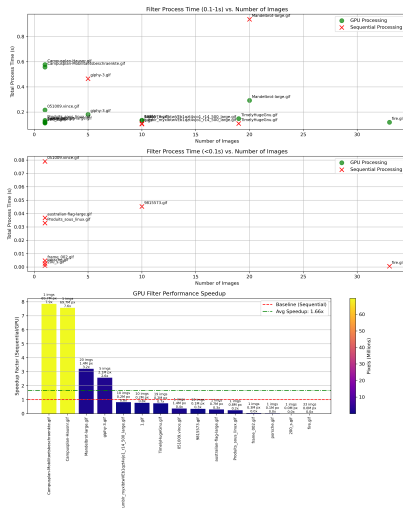


Figure: Runtime CUDA Version

Master-Worker Model:

- Master: Data handling
- Workers: Independent processing

Hybrid MPI+OpenMP:

- MPI: Distributed workload
- OpenMP: Node-level parallelism

MPI+OpenMP Performance

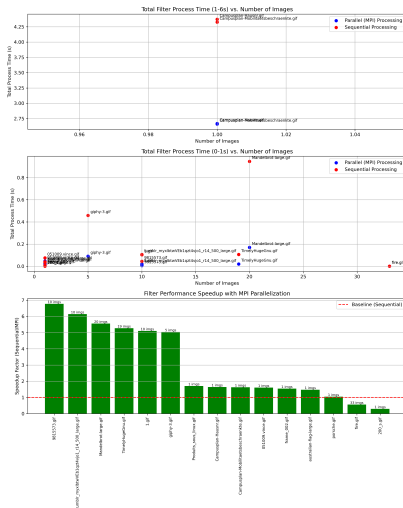


Figure: Runtime MPI+OpenMP Version

Hybrid Implementation

- MPI for Sobel filter (best improvement)
- CUDA for blur filter with large images
- OpenMP for thread-level parallelism

Adaptive parallelization based on workload.

Hybrid Performance

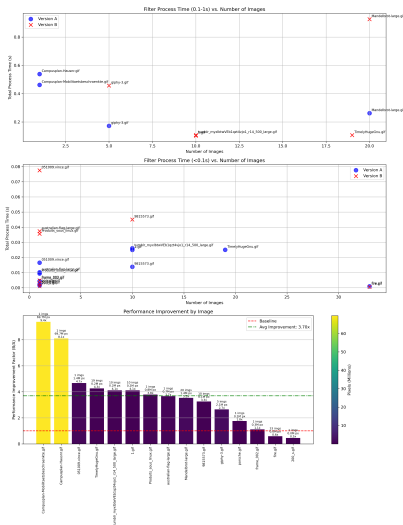


Figure: Runtime Hybrid MPI+OpenMP+CUDA Version

Conclusion

- Adaptive parallelization beneficial
- CUDA optimal for intensive filters
- Effective hybrid MPI+OpenMP + CUDA strategy

Thank you!